UNIVERSIDAD AMERICANA



PROGRAMACIÓN DE APLICACIONES DE ESCRITORIO

DOCUMENTO TÉCNICO

CasosDeEstudioProyecto | Evaluaciones integradas 1, 2 y 3

RESERVAS · PEDIDOS · CAJA DE VENTAS

INTEGRANTES	Zahid Benjamín Reyes López, José Alessandro Torres Parodi
DOCENTE	Freddy López
FECHA	28/10/2026
REPOSITORIO	github.com/rzahidb/CasosDeEstudio · rama main

01 / PRESENTACIÓN

Informe técnico integral de los tres casos

Documento unificado: requisitos, código importante, diseño JavaFX y registro de evidencias de las tareas 1, 2 y 3.

El proyecto CasosDeEstudioProyecto reúne tres evaluaciones prácticas de Programación de Aplicaciones de Escritorio de la Universidad Americana. Este informe se diseñó como documento único de entrega, no como índice de informes separados. Se concentra en cómo se desarrollaron las simulaciones, cómo se conectaron las vistas FXML con sus controllers y cómo los services administran los hilos sin bloquear la interfaz.

1.1 Objetivo general

Desarrollar y documentar una aplicación de escritorio en JavaFX capaz de ejecutar reservas concurrentes, procesar pedidos con cancelación y registrar ventas concurrentes con consistencia de datos, utilizando clases Java, Threads, Runnable, synchronized e interrupt().

1.2 Alcance y evaluación

Tarea sumativa	Proceso	Resultado de referencia	Valor
1 · Reserva de entradas	3 agentes × 10 intentos; inventario de 20	20 confirmadas / 10 rechazadas / 0 disponibles	40 pts
2 · Procesamiento de pedidos	30 pedidos secuenciales × 200 ms	30/30, 100 %, Completado; o cancelación parcial	40 pts
3 · Caja de ventas	3 vendedores × 5 ventas de C$100	15 ventas / C$1,500; cancelación consistente	40 pts

Los valores anteriores son resultados esperados definidos por las rúbricas. Las tablas de ejecución y los espacios de captura incluidos más adelante deberán completarse con resultados observados realmente en IntelliJ IDEA. El documento no reemplaza la demostración ni presume pruebas realizadas.

1.3 Entorno del proyecto

Elemento	Configuración verificada en los archivos del repositorio
Repositorio	https://github.com/rzahidb/CasosDeEstudio/tree/zahid
Nombre Maven	CasosDeEstudioProyecto
Paquete raíz	org.example.casosdeestudioproyecto
Tecnologías	Java 21; JavaFX 21.0.6; Maven; JavaFX Controls/FXML
Lanzamiento	Launcher → MainApplication → ViewNavigator → menu-view.fxml
Paquetes empleados	controllers, model, services, util
Ubicación FXML	src/main/resources/org/example/casosdeestudioproyecto/views /
Comando de ejecución	mvn clean javafx:run

02 / ARQUITECTURA

Cómo se organiza la aplicación

Una única aplicación JavaFX; no se crean tres aplicaciones independientes.

2.1 Estructura de paquetes

Elemento	Ejemplos de archivos	Responsabilidad principal
views (FXML)	menu, reservas, pedidos, caja	Define los controles visibles y sus identificadores fx:id.
controllers	MenuController; ReservasController; PedidosController; CajaController	Recibe clics, actualiza etiquetas y barras, administra estados visuales.
model	InventarioEntradas; AgenteReserva; ProcesadorPedidos; Caja; Vendedor; Resultado...	Representa datos, resultados y trabajadores Runnable.
services	ReservasService; PedidosService; CajaService	Inicia/coordina hilos, evita duplicados, cancela, espera finalización y notifica progreso.
util	ViewNavigator	Cambia de vista y coordina el retorno al menú/cierre.
raíz	Launcher; MainApplication; module-info	Arranca JavaFX y declara el módulo y apertura FXML.

2.2 Recorrido desde el botón hasta la barra

1. La vista FXML: Declara un Button con onAction y un ProgressBar con fx:id.

2. El controller: JavaFX inyecta el control en @FXML; el método del botón llama al service.

3. El service: Crea uno o varios trabajadores Runnable, genera Threads y entrega callbacks.

4. El model / trabajador: Procesa una operación y produce los nuevos contadores o una instantánea.

5. Platform.runLater: El service envía la notificación al JavaFX Application Thread.

6. El controller actualiza: Calcula fracción completada/total y llama a setProgress() en la barra.

2.3 Qué NO hace cada capa

- El FXML no ejecuta Thread.sleep() ni modifica inventarios: solo define controles y vincula eventos.
- El controller no debe bloquear la interfaz con join() o sleep(); su trabajo es manejar la experiencia visual.
- El service no dibuja barras directamente: comunica datos que el controller muestra mediante los controles inyectados.
- Los models no necesitan conocer un botón, Label o ProgressBar de JavaFX.

2.4 Inicialización y cierre

Launcher.main ejecuta Application.launch(MainApplication.class, args); MainApplication configura el Stage mediante ViewNavigator. Las vistas se cargan con FXMLLoader y los controladores usan initialize() para establecer etiquetas y barras en cero. Los services registran manejadores de detención para salir de una vista sin dejar tareas activas. Para comprobarlo en la defensa se requiere mostrar el cierre durante una simulación, no únicamente la apariencia del botón.

03 / ELEMENTO CENTRAL

Cómo añadimos las barras de progreso

Las barras son controles reales de JavaFX: ProgressBar, con un valor double desde 0.0 (vacío) hasta 1.0 (completo).

3.1 Paso 1: declarar el control visual en FXML

En el archivo FXML se importa javafx.scene.control.ProgressBar y se añade el control donde se necesita. El fx:id funciona como el vínculo entre esa barra y el campo Java del controller. El atributo progress="0" define su estado visual inicial.

CÓDIGO 01 · BARRA DECLARADA EN EL FXML

src/main/resources/.../views/reservas-view.fxml (extracto)

```
<?import javafx.scene.control.ProgressBar?>
<ProgressBar fx:id="barraProgreso"
             prefHeight="20" progress="0"/>
```

3.2 Paso 2: conectar fx:id con @FXML

CÓDIGO 02 · CAMPO DEL CONTROLLER

src/main/java/.../controllers/ReservasController.java

```
@FXML
private ProgressBar barraProgreso;
```

El nombre barraProgreso debe ser idéntico en ambos archivos. FXMLLoader inyecta la instancia creada por el FXML y permite que el controller invoque métodos del control. Si el campo es distinto o no existe, la vista puede no cargar o fallar al manipular el valor.

3.3 Paso 3: inicializar y actualizar

CÓDIGO 03 · VALORES QUE CONTROLA JAVAFX

ReservasController.java (fragmentos de dos métodos)

```
// En limpiarCampos(), al comenzar una nueva simulación:
barraProgreso.setProgress(0.0);


// Cuando cambia el contador:
barraProgreso.setProgress((double) procesadas
        / ReservasService.SOLICITUDES_TOTALES);
```

No se envía el número 20, 30 o 100 a la barra. Se le envía una fracción: por ejemplo, 10 solicitudes de 30 = 0.3333 (33.3 %). El cast a double evita la división entera. Al llegar a 30/30, la fracción vale 1.0 y la barra aparece completa.

3.4 Paso 4: recibir los datos sin bloquear JavaFX

CÓDIGO 04 · NOTIFICACIÓN AL HILO GRÁFICO

ReservasService.java (extracto simplificado de callback)

```
Platform.runLater(() -> {
    if (idSimulacion == obtenerIdSimulacionActual()) {
        alActualizarIntento.accept(resultado);
    }
});
```

Los Threads no modifican directamente la ProgressBar; el service usa Platform.runLater para que el controller reciba el resultado en el hilo JavaFX. El identificador de simulación impide que una notificación pendiente de una ejecución anterior se aplique a otra.

3.5 Fórmulas por pantalla

ReservasController: barraProgreso = solicitudes procesadas/30. PedidosController: barraProgreso = pedidos completados/30. CajaController: barraGeneral = ventas totales/15 y barras de vendedores 1, 2 y 3 = ventas individuales/5.

04 / SERVICES

Para qué sirven los servicios y cómo los utilizamos

Los services son el puente seguro entre los eventos de los controllers y los hilos del modelo.

4.1 Responsabilidades que comparten

Controlar el inicio Verifican enEjecucion; si ya existe una simulación no crean Threads duplicados.

Construir trabajadores Instancian AgenteReserva, ProcesadorPedidos o Vendedor para cada nueva ejecución.

Ejecutar en segundo plano Crean Thread y llaman start(), manteniendo libre el JavaFX Application Thread.

Informar cambios Exponen callbacks (Consumer/BiConsumer) y emplean Platform.runLater para notificar a controllers.

Cancelar correctamente Envían interrupt() y permiten que los Runnable salgan por comprobación o InterruptedException.

Esperar de forma asíncrona Utilizan un hilo coordinador con join() antes de habilitar reinicio o navegación.

Evitar actualizaciones atrasadas Asocian notificaciones a un identificador de ejecución o simulación.

4.2 Comparación de los tres services

Service	Hilos trabajadores	Recurso compartido	Cómo informa al controller
ReservasService	3 hilos AgenteReserva	InventarioEntradas	ResultadoReservas y resumen de agentes
PedidosService	1 hilo ProcesadorPedidos	Contador del procesador	cantidad de pedidos, mensajes y EstadoProcesamiento
CajaService	3 hilos Vendedor	Caja	ResultadoCaja, int[] con ventas por vendedor

4.3 Ejemplo del service enviando un evento

CÓDIGO 05 · AGENTE + DEVOLUCIÓN DE PROGRESO

`ReservasService.java (extracto condensado; callbacks null-check omitidos)`

```
agentes[i] = new AgenteReserva(indice, nombre,
        INTENTOS_POR_AGENTE, inventario,
        (idx, resultado) -> {
    String resumen = construirResumenAgentes();
    Platform.runLater(() -> {
        if (idSimulacion == obtenerIdSimulacionActual()) {
            alActualizarIntento.accept(resultado);
            alActualizarResumenAgentes.accept(resumen);
        }
    });
});
```

El service recibe los resultados generados por el trabajador; no conoce el Label ni la ProgressBar concretos. Esa notificación termina llamando a actualizarVistaParcial() en ReservasController, gracias a la referencia this::actualizarVistaParcial pasada al iniciar.

4.4 Por qué join() se hace fuera del controller

CÓDIGO 06 · COORDINACIÓN SIN CONGELAR LA VENTANA

`ReservasService.java (extracto explicativo)`

```
Thread coordinador = new Thread(() -> {
    esperarHilos(hilosEstaEjecucion);
    ResultadoReservas resultado = inventario.obtenerEstadoActual();
    // El service marca finalización y notifica con runLater.
}, "Coordinador-Reservas");
coordinador.start();
```

Si se ejecutara join() al pulsar un botón del controller, la ventana podría congelarse mientras se esperan los hilos. Por eso los services realizan esa espera en un Thread separado y solo entonces notifican que la simulación terminó.

05 / CASO 1

Reserva concurrente de entradas

Tarea sumativa 1 · IL3.1 e IL3.2 · 40 puntos

5.1 Propósito y parámetros

Se simula la reserva de entradas para un evento. Existen 20 entradas al inicio; tres agentes realizan diez intentos cada uno (30 solicitudes). Cada reserva confirmada consume una entrada. Una vez agotadas las existencias, los intentos posteriores se registran como rechazados.

Configuración	Valor
Recurso compartido	Una instancia de InventarioEntradas
Trabajadores	Tres AgenteReserva que implementan Runnable
Solicitudes	3 × 10 = 30
Resultado esperado completo	20 confirmadas; 10 rechazadas; 0 entradas
Regla de consistencia	disponibles + confirmadas == 20; disponibles >= 0

5.2 Archivo FXML y controller responsable de la barra

CÓDIGO 07 · FXML (ELEMENTOS PRINCIPALES)

`views/reservas-view.fxml · fx:controller=...controllers.ReservasController`

```
<ProgressBar fx:id="barraProgreso"
             maxWidth="1.7976931348623157E308"
             prefHeight="20" progress="0"/>
<Button fx:id="btnIniciar"
        text="Iniciar Simulación"
        onAction="#iniciarSimulacion"/>
```

CÓDIGO 08 · AVANCE DE SOLICITUDES

`ReservasController.java (extracto con constantes numéricas sustituidas por su valor: 30)`

```
private void actualizarVistaParcial(ResultadoReservas resultado) {
    lblEntradasDisponibles.setText(String.valueOf(
            resultado.getEntradasDisponibles()));
    lblReservasConfirmadas.setText(String.valueOf(
            resultado.getReservasConfirmadas()));
    lblSolicitudesRechazadas.setText(String.valueOf(
            resultado.getSolicitudesRechazadas()));
    int procesadas = resultado.getTotalProcesadas();
    lblTotalSolicitudes.setText(procesadas + " / 30");
    barraProgreso.setProgress((double) procesadas / 30);
    agregarRegistro(resultado.getDetalleOperacion());
}
```

Cada vez que un agente termina un intento, el service entrega un ResultadoReservas al controller. El método actualiza tres contadores, el total procesado, la barra y el registro de operaciones. Se cuentan tanto intentos confirmados como rechazados porque ambos se procesaron.

5.3 Bloque sincronizado del inventario

CÓDIGO 09 · SECCIÓN CRÍTICA DE INVENTARIOENTRADAS

`model/InventarioEntradas.java (extracto; omite formato de mensaje y return)`

```
public synchronized ResultadoReservas reservarEntrada(
        String nombreAgente, int numeroSolicitud) {
    boolean confirmada;
    if (entradasDisponibles > 0) {
        entradasDisponibles--;
        reservasConfirmadas++;
        confirmada = true;
    } else {
        solicitudesRechazadas++;
        confirmada = false;
    }
    // Después construye el mensaje y devuelve ResultadoReservas.
}
```

La palabra synchronized evita que dos agentes lean y descuenten la misma última entrada al mismo tiempo. La verificación y la modificación se encuentran protegidas por el mismo bloqueo: no son dos operaciones

independientes. La espera Thread.sleep(160) se realiza dentro de AgenteReserva, fuera de este método sincronizado.

5.4 ¿Qué hace ReservasService?

- Crea tres instancias de AgenteReserva y tres Threads; todos utilizan el mismo inventario.
- Previene el inicio duplicado mediante enEjecucion.
- Recibe el progreso y utiliza Platform.runLater antes de actualizar el controller.
- Al concluir los trabajadores, un coordinador los espera con join() y entrega el resumen final.
- Al navegar fuera de la pantalla, solicita interrupción y espera de forma asíncrona.

5.5 Secuencia que debe explicarse al profesor

Iniciar Simulación → ReservasController.iniciarSimulacion() → ReservasService.iniciarSimulacion() → tres AgenteReserva.run() → InventarioEntradas.reservarEntrada() → ResultadoReservas → Platform.runLater() → ReservasController.actualizarVistaParcial() → barraProgreso.setProgress().

06 / CASO 1 · PRUEBAS

Resultados y evidencias de reservas

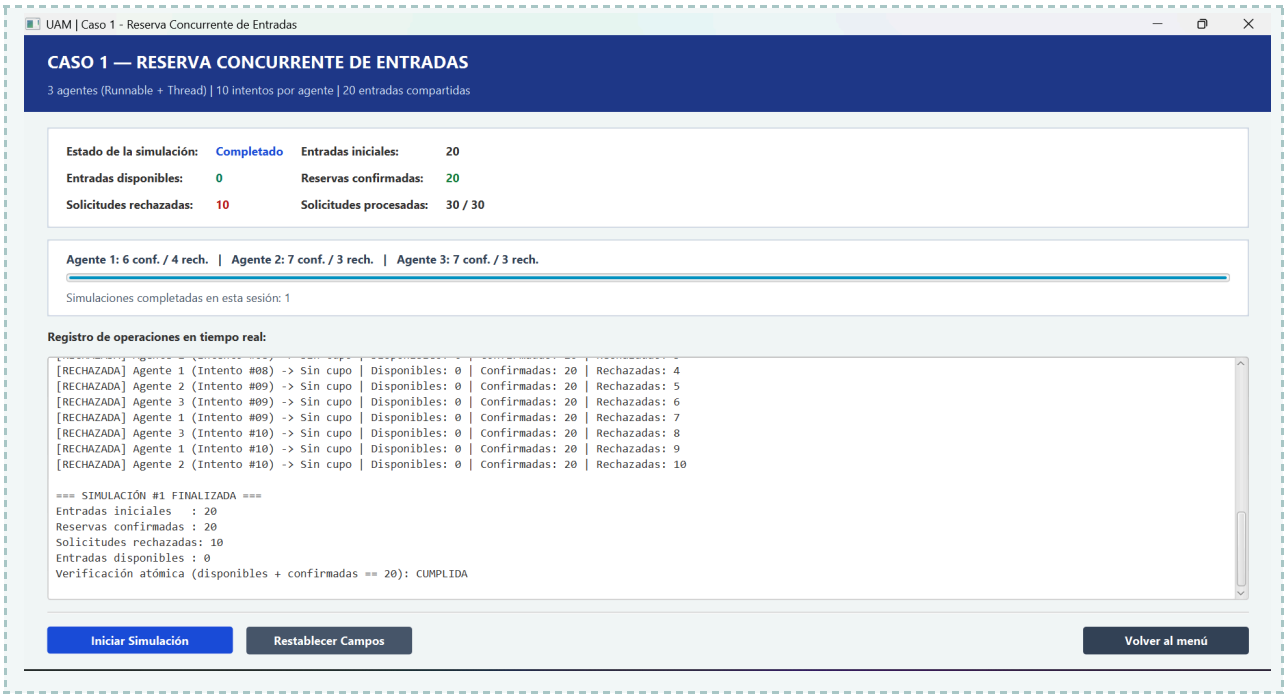
La rúbrica exige cinco ejecuciones completas independientes, además del rechazo de duplicados y la coherencia del inventario.

Prueba	Confirmadas (esperado)	Rechazadas (esperado)	Disponibles (esperado)	Observado / fecha
R1	20	10	0	Correcto
R2	20	10	0	Correcto
R3	20	10	0	Correcto
R4	20	10	0	Correcto
R5	20	10	0	Correcto

6.1 Procedimiento de aceptación

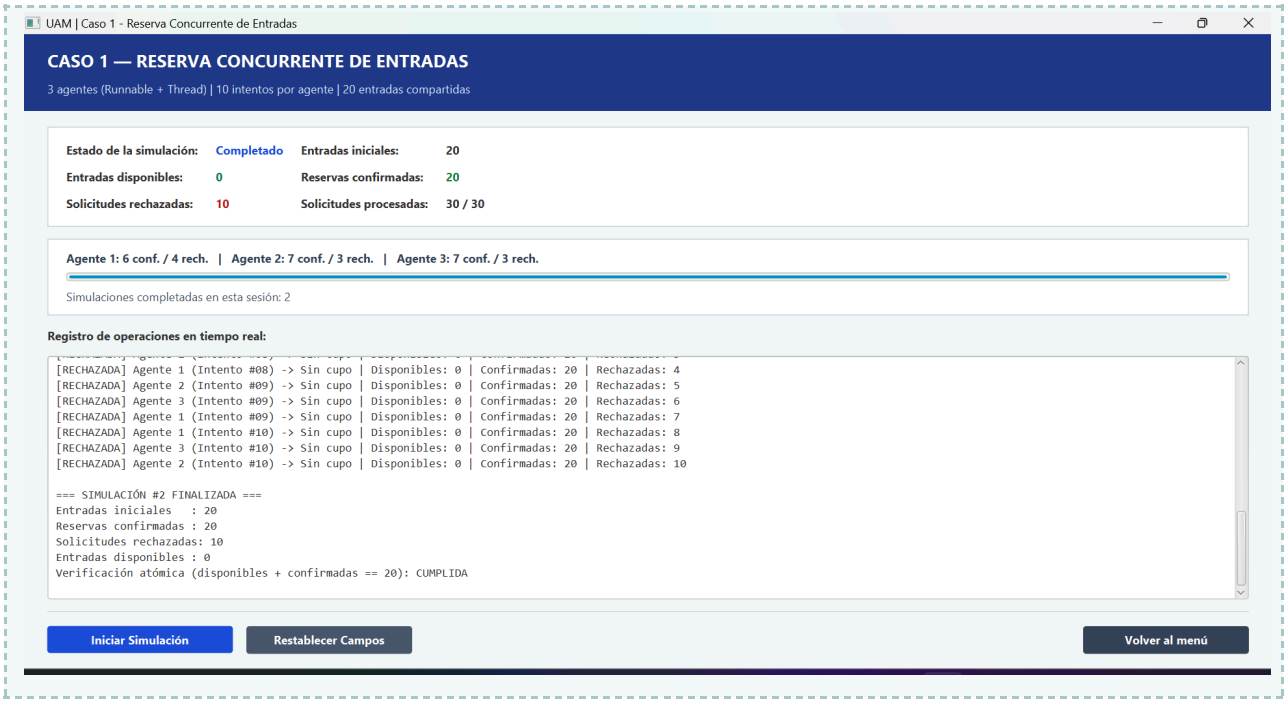
- Ejecutar Iniciar Simulación cinco veces; esperar “Completado” antes de iniciar otra ejecución.
- Comprobar en cada intento que confirmadas + disponibles = 20 y que disponibles nunca baja de cero.
- Intentar iniciar otra simulación mientras se ejecuta la anterior; no deben crearse agentes extra.
- Observar registro, barra y etiquetas mientras hay actividad; la ventana debe responder.
- Volver al menú durante una ejecución y explicar cómo el service finaliza los trabajadores.

EVIDENCIA R1 · EJECUCIÓN COMPLETA NÚMERO 1

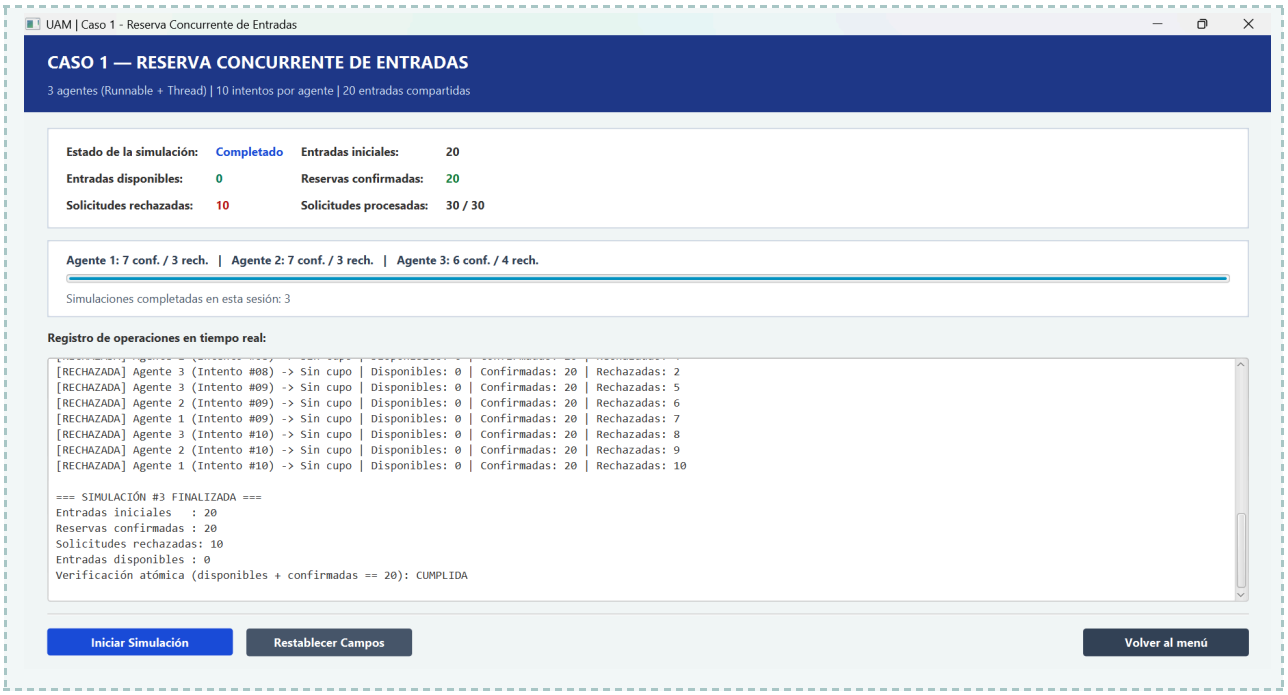


EVIDENCIA R2 · EJECUCIÓN COMPLETA NÚMERO 2

Captura que debes insertar: Capturar la pantalla de Reservas al finalizar la ejecución 2, con 20 confirmadas, 10 rechazadas, 0 disponibles, barra completa y contador de pruebas.

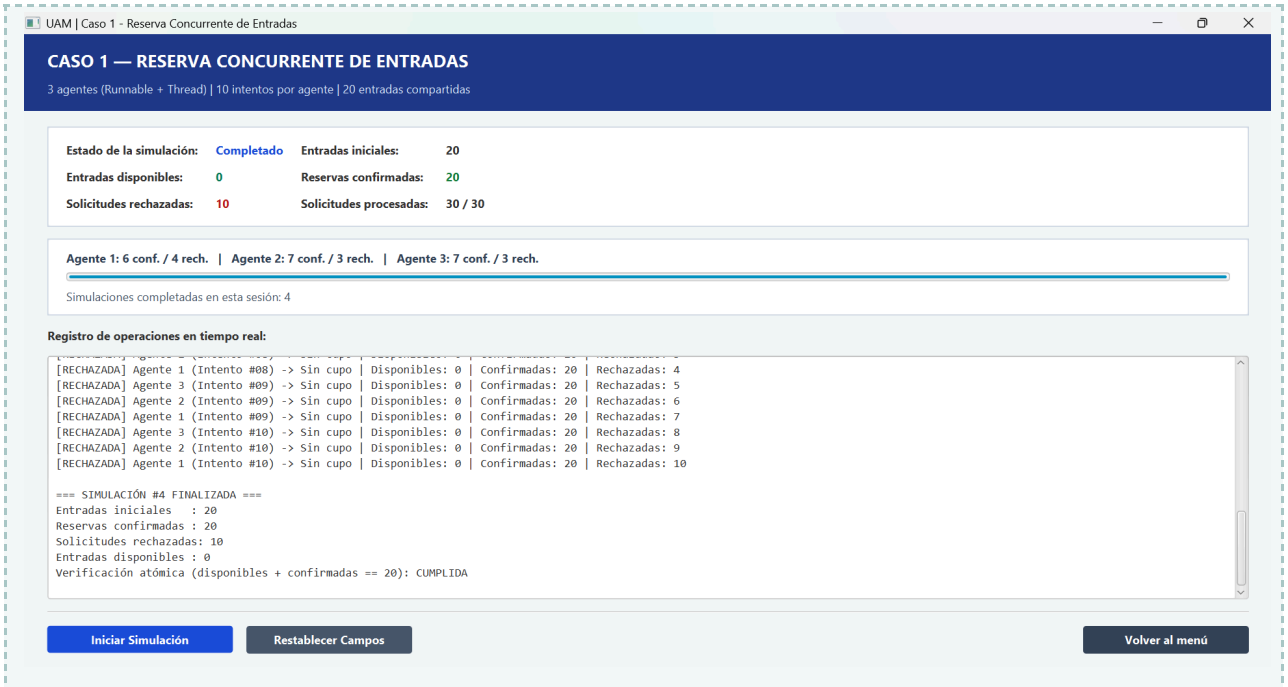


EVIDENCIA R3 · EJECUCIÓN COMPLETA NÚMERO 3



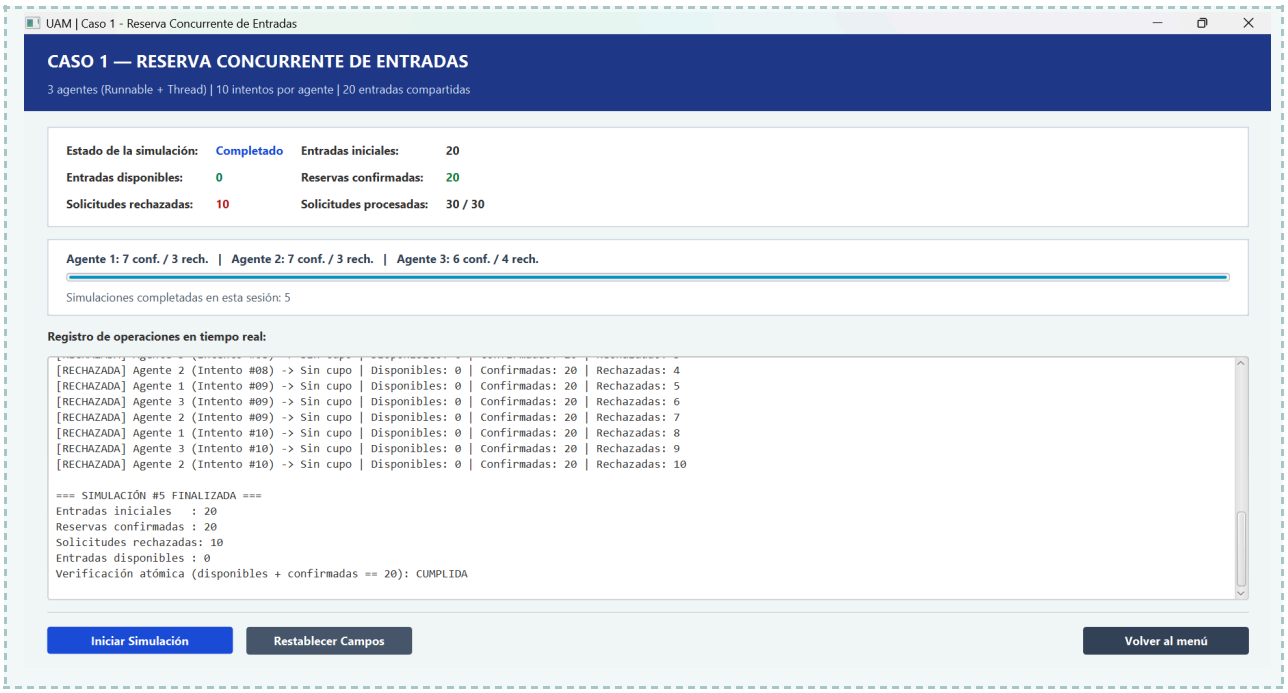
EVIDENCIA R4 · EJECUCIÓN COMPLETA NÚMERO 4

Captura que debes insertar: Capturar la pantalla de Reservas al finalizar la ejecución 4, con 20 confirmadas, 10 rechazadas, 0 disponibles, barra completa y contador de pruebas.



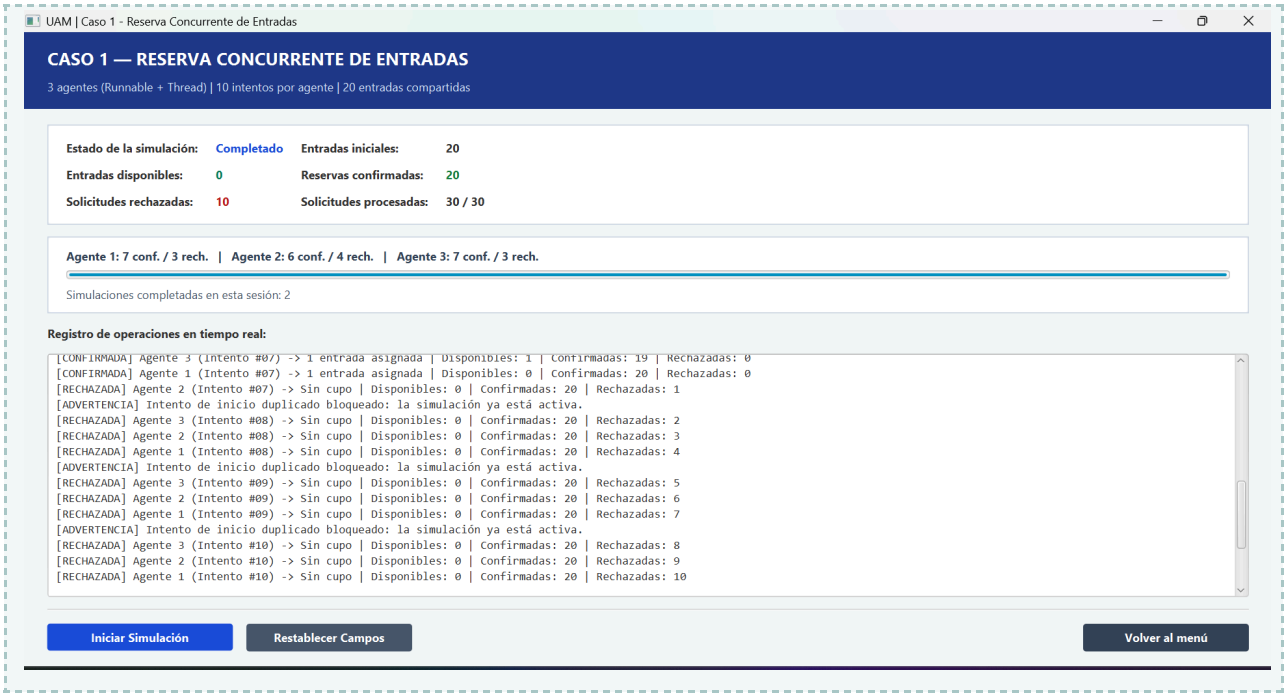
EVIDENCIA R5 · EJECUCIÓN COMPLETA NÚMERO 5

Captura que debes insertar: Capturar la pantalla de Reservas al finalizar la ejecución 5, con 20 confirmadas, 10 rechazadas, 0 disponibles, barra completa y contador de pruebas.

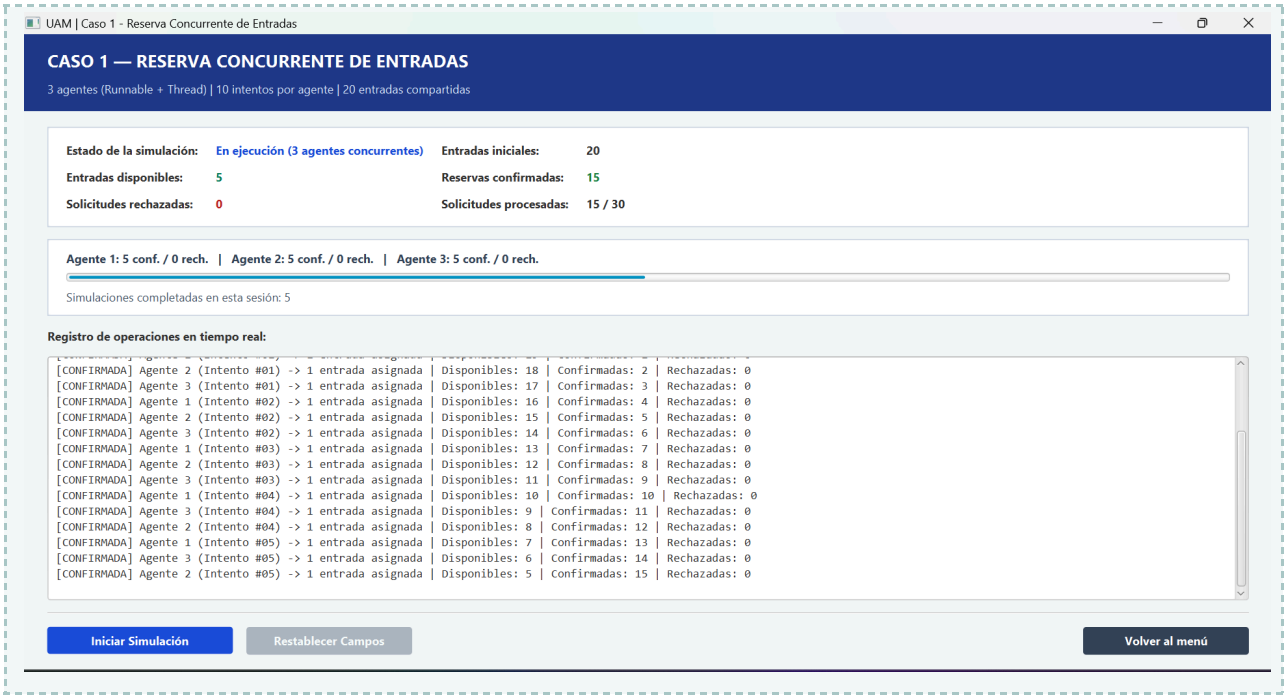


EVIDENCIA R6 · INICIO DUPLICADO Y RESPUESTA DE LA INTERFAZ

Captura que debes insertar: Capturar la advertencia de intento duplicado en el registro, junto con los contadores y la barra en ejecución.



EVIDENCIA R7 · CONCURRENCIA Y ACTUALIZACIÓN EN TIEMPO REAL



07 / CASO 2

Procesamiento secuencial de pedidos con cancelación

Tarea sumativa 2 · IL3.1 e IL3.3 · 40 puntos

7.1 Propósito y parámetros

Un solo ProcesadorPedidos implementa Runnable para completar hasta 30 pedidos, uno después de otro. Cada pedido simula un trabajo de 200 milisegundos. Se usa Thread para mantener la interfaz operativa; cancelar no elimina a la fuerza el hilo, sino que solicita su interrupción.

Aspecto	Comportamiento requerido
Cantidad de trabajadores	1 por ejecución
Pedidos	30; procesados de forma secuencial
Tiempo simulado	Thread.sleep(200) en cada pedido
Finalización completa	30/30, barra 100 %, estado Completado
Cancelación parcial	Conserva solamente pedidos completados; estado Cancelado
Estados	Listo; En ejecución; Cancelando; Cancelado; Completado; Error

7.2 ¿Qué archivo contiene la barra?

CÓDIGO 10 · CONTROLES DEL FXML

`views/pedidos-view.fxml · fx:controller=...controllers.PedidosController`

```
<ProgressBar fx:id="barraProgreso"
             maxWidth="1.7976931348623157E308"
             prefHeight="22" progress="0"/>
<Button fx:id="btnCancelar" text="Cancelar"
        onAction="#cancelarProcesamiento"/>
```

7.3 Cómo se actualizan contador, porcentaje y barra

CÓDIGO 11 · ACTUALIZACIÓN DE LA BARRA DEL CASO 2

`controllers/PedidosController.java (método de la rama zahid)`

```
private void actualizarProgresoPedido(
        int pedidosCompletados, String mensaje) {
    lblContadorPedidos.setText(pedidosCompletados
            + " / " + PedidosService.TOTAL_PEDIDOS);
    double fraccion = (double) pedidosCompletados
            / PedidosService.TOTAL_PEDIDOS;
    int porcentaje = (int) Math.round(fraccion * 100);
    barraProgreso.setProgress(fraccion);
    lblPorcentaje.setText(porcentaje + " %");
    agregarRegistro(mensaje);
}
```

El controller recibe pedidosCompletados después de que el trabajador confirma cada pedido. La barra muestra pedidosCompletados/30, y lblPorcentaje multiplica esa misma fracción por 100. Si se interrumpe durante una espera, el pedido no se suma y la barra conserva el valor parcial.

7.4 Código de cancelación cooperativa

CÓDIGO 12 · MANEJO DE INTERRUPTEDEXCEPTION

`model/ProcesadorPedidos.java (extracto)`

```
try {
    Thread.sleep(duracionPorPedidoMs);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    marcarCancelado("Pedido #" + numeroPedido
            + " interrumpido; no se contabiliza.");
    return;
}
```

El incremento pedidosCompletados++ ocurre después de la espera y de comprobar isInterrupted(); un pedido cancelado antes de finalizar no se cuenta.

08 / CASO 2 · SERVICES

Estados, reinicio y cierre del trabajador

El requisito central no es solo enviar interrupt(), sino comprobar que el hilo terminó antes de permitir otro inicio.

8.1 Funciones de PedidosService

iniciarProcesamiento(...) Marca el estado En ejecución, construye un ProcesadorPedidos y un Thread nuevo, arranca el trabajo y crea un coordinador.

solicitarCancelacion(...) Cambia a Cancelando cuando corresponde, llama interrupt() y publica los mensajes/estados de forma segura.

detenerYEsperarAsincrono(...) Solicita la detención al salir o cerrar la ventana y espera al trabajador sin bloquear la interfaz.

esperarHilo(...) Utiliza join() en un hilo secundario para confirmar la finalización efectiva.

CÓDIGO 13 · CALLBACK DE PROGRESO DE PEDIDOS

`services/PedidosService.java (argumento del constructor ProcesadorPedidos)`

```
(completados, mensaje) -> Platform.runLater(() -> {
    if (id == obtenerIdEjecucionActual()
            && alCompletarPedido != null) {
        alCompletarPedido.accept(completados, mensaje);
    }
}),
```

El callback mostrado es parte del constructor de ProcesadorPedidos en el service. Los callbacks adicionales de cancelación y error se encuentran en el archivo completo; el progreso llega al controller a través de Platform.runLater, después de comprobar el identificador de ejecución.

8.2 Estados de la pantalla

Estado	Cuándo se utiliza	Acción visible
Listo	Antes de comenzar o después de restablecer	0/30, 0 %, Iniciar disponible
En ejecución	Se está procesando el lote	Avanza contador y barra
Cancelando	Se solicitó interrupt()	Se bloquea nuevo inicio mientras se espera
Cancelado	El coordinador confirma fin de ejecución incompleta	Se conserva el conteo parcial
Completado	Finalizaron los 30 pedidos	30/30, 100 %
Error	Hubo fallo capturado durante el proceso	La aplicación informa el estado

8.3 Pruebas y resultados

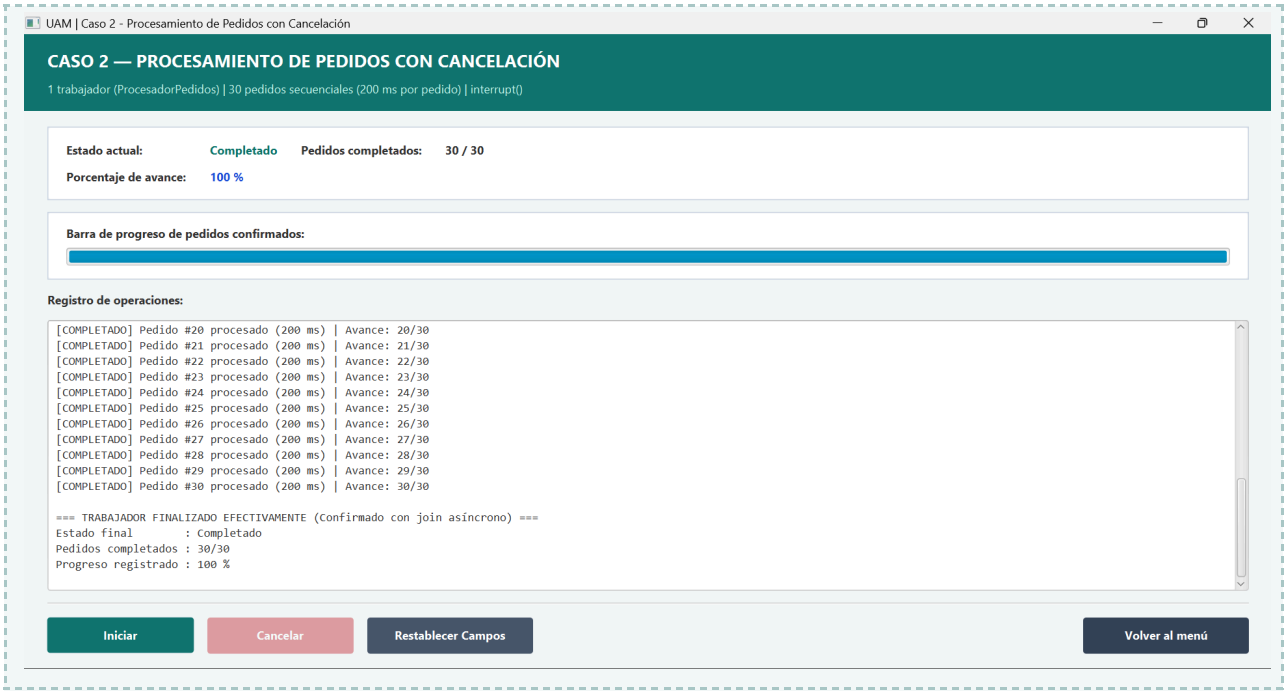
ID	Escenario	Resultado esperado	Resultado real
P1	Lote completo	30/30 · 100 % · Completado	Completado
P2	Cancelar durante una espera	Cancelado, contador parcial estable	Cancelado, contador parcial estable
P3	Inicio duplicado	Rechazado o impedido	Rechazado
P4	Reinicio	Comienza 0/30; nuevo Thread	Nuevo Thread
P5	Cerrar con tarea activa	Termina el trabajador sin congelar JavaFX	Termina sin congelar JavaFX

09 / CASO 2 · CAPTURAS

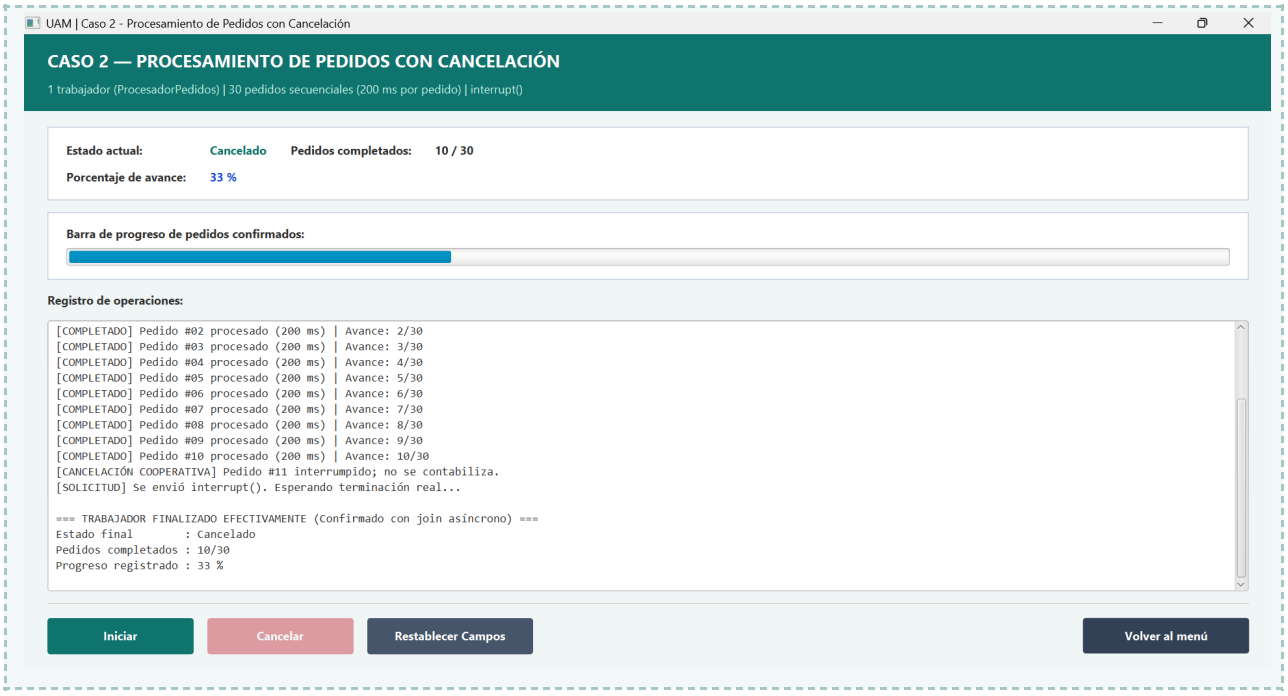
Evidencias del procesamiento de pedidos

Insertar capturas de IntelliJ y de la pantalla JavaFX según cada escenario de aceptación.

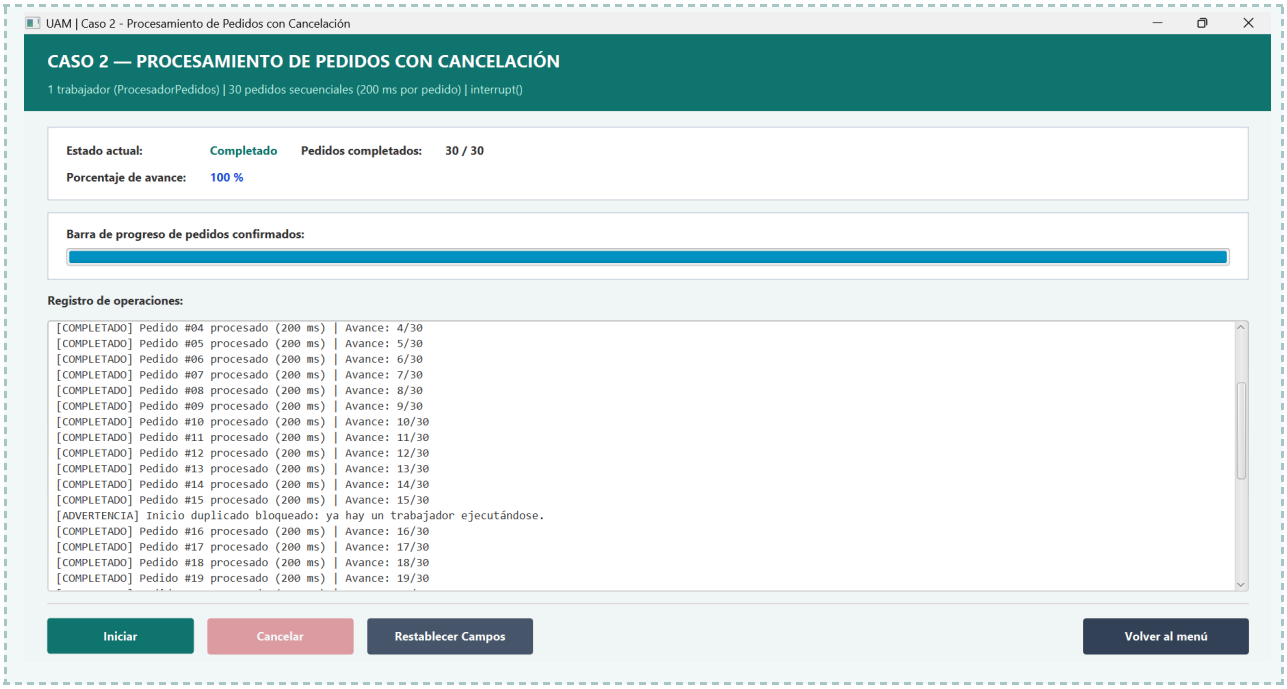
EVIDENCIA P1 · EJECUCIÓN COMPLETA



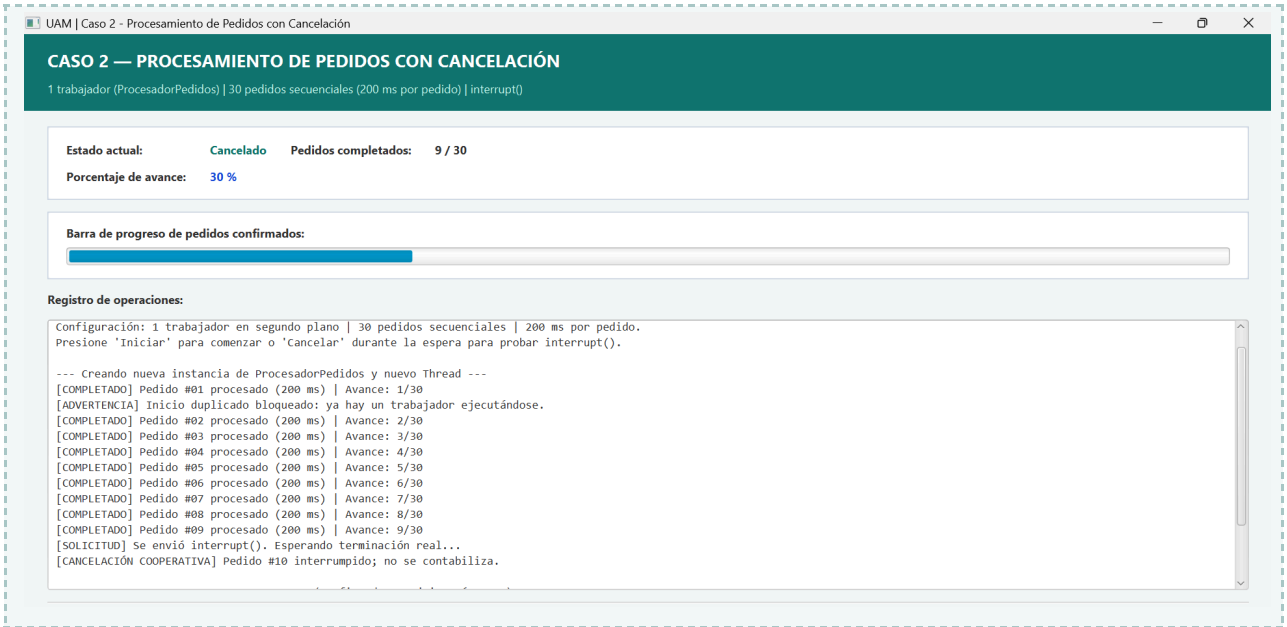
EVIDENCIA P2 · CANCELACIÓN PARCIAL



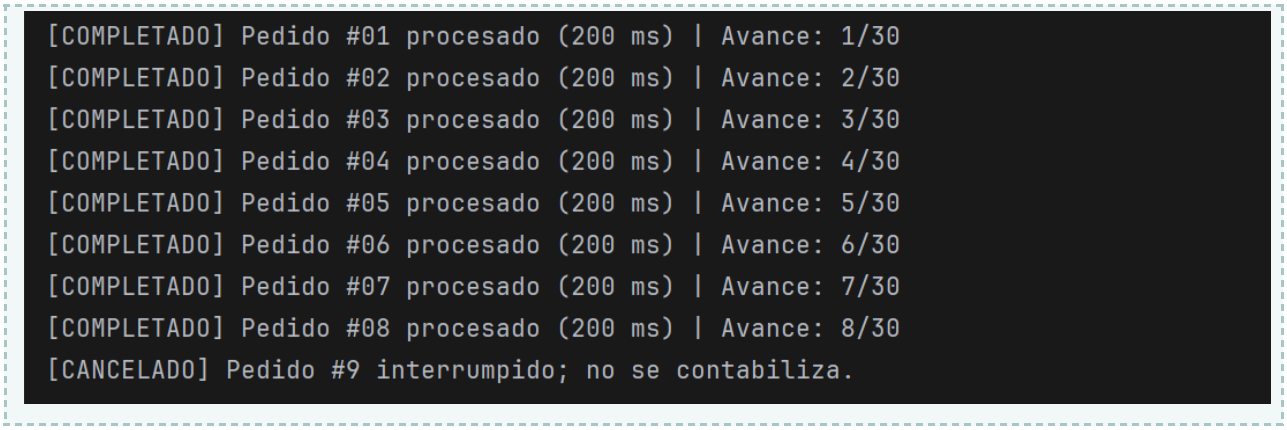
EVIDENCIA P3 · INICIO DUPLICADO



EVIDENCIA P4 · REINICIO DESPUÉS DE CANCELAR



EVIDENCIA P5 · CIERRE CON TAREA ACTIVA



10 / CASO 3

Caja de ventas concurrente con control de hilos

Tarea sumativa 3 · IL3.1, IL3.2 e IL3.3 · 40 puntos

10.1 Propósito y parámetros

Tres vendedores, cada uno implementado como Runnable y ejecutado en su propio Thread, realizan cinco ventas de C$100. Comparten la misma instancia de Caja. La simulación muestra el avance de cada vendedor, las ventas generales y la recaudación, con posibilidad de cancelación y reinicio seguros.

Parámetro	Valor esperado
Vendedores	3
Ventas por vendedor	5
Monto por venta	C$100
Total al completar	15 ventas; C$1,500
Regla constante	totalRecaudado == cantidadVentas * 100
Delay de simulación	Thread.sleep(450) por intento en Vendedor

10.2 Cuatro barras en un solo FXML

CÓDIGO 14 · BARRAS GENERALES E INDIVIDUALES

views/caja-view.fxml (extracto de cuatro controles)

```
<ProgressBar fx:id="barraGeneral" progress="0"/>
<ProgressBar fx:id="barraVendedor1" progress="0"/>
<ProgressBar fx:id="barraVendedor2" progress="0"/>
<ProgressBar fx:id="barraVendedor3" progress="0"/>
```

No utilizamos una sola barra para los tres trabajadores. caja-view.fxml define cuatro ProgressBar: la general representa las 15 ventas totales y cada barra individual representa las cinco ventas de su vendedor. Todos los controles están vinculados por fx:id con campos @FXML del CajaController.

10.3 Cómo CajaController calcula cada avance

CÓDIGO 15 · BARRA GENERAL Y CIFRAS ACUMULADAS

controllers/CajaController.java · dentro de actualizarAvanceVentas()

```
int ventasTotales = resultadoCaja.getCantidadVentas();
int recaudadoTotal = resultadoCaja.getTotalRecaudado();
lblTotalVentas.setText(ventasTotales + " / "
        + CajaService.TOTAL_VENTAS_ESPERADAS);
lblTotalRecaudado.setText(formatearCordobas(recaudadoTotal));
barraGeneral.setProgress((double) ventasTotales
        / CajaService.TOTAL_VENTAS_ESPERADAS);
```

CÓDIGO 16 · BARRA INDIVIDUAL DE CADA VENDEDOR

controllers/CajaController.java (método de la rama zahid)

```
private void actualizarControlVendedor(
        int numero, int ventas, Label etiqueta, ProgressBar barra) {
    int monto = ventas * CajaService.MONTO_POR_VENTA;
    etiqueta.setText(ventas + " / " +
            CajaService.VENTAS_POR_VENDEDOR + " ventas (" +
            formatearCordobas(monto) + ")");
    barra.setProgress((double) ventas
            / CajaService.VENTAS_POR_VENDEDOR);
}
```

Ejemplo: si Vendedor 2 registra 3 ventas, su barra vale 3/5 = 0.60; al mismo tiempo, la barra general usa el total de ventas confirmado por la caja, dividido entre 15.

11 / CASO 3 · SINCRONIZACIÓN

Por qué contador y dinero se actualizan juntos

La caja debe asegurar que ventas y recaudación describan exactamente las mismas operaciones.

11.1 Registro atómico con synchronized

CÓDIGO 17 · ACTUALIZACIÓN SIMULTÁNEA DEL RECURSO COMPARTIDO

model/Caja.java (método de la rama zahid)

```
public synchronized ResultadoCaja registrarVentas(
        String vendedor, int monto) {
    cantidadVentas++;
    totalRecaudado += monto;
    String mensaje = vendedor + " registró una venta de C$" + monto
            + " | Ventas acumuladas: " + cantidadVentas
            + " | Total: C$" + totalRecaudado;
    System.out.println(mensaje);
    return new ResultadoCaja(cantidadVentas,
            totalRecaudado, mensaje);
}
```

El método synchronized protege dos variables relacionadas durante una misma venta. Si varios vendedores acceden a la caja al mismo tiempo, solo uno completa el registro dentro de esa sección crítica. Una venta confirmada incrementa una unidad y C$100; por ello se conserva la relación totalRecaudado = cantidadVentas × 100.

11.2 Qué ocurre cuando se cancela a un vendedor

CÓDIGO 18 · TRABAJO FUERA DE LA SECCIÓN CRÍTICA

model/Vendedor.java (extracto; chequeo adicional de interrupción omitido)

```
try {
    Thread.sleep(450);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    notificarInterrupcion(nombreVendedor
            + " fue interrumpido durante la venta #" + numero);
    return;
}
ResultadoCaja resultado = cajaCompartida.registrarVentas(
        nombreVendedor, montoPorVenta);
ventasRealizadas++;
```

El trabajador comprueba las interrupciones antes y después de la espera. Si la tarea se interrumpe durante sleep(), no se registra esa venta. Si ya entró en el método sincronizado, la actualización del contador y el dinero se completa junta; no debe quedar una venta a medias.

11.3 Qué hace CajaService

- Crea una Caja compartida, tres objetos Vendedor y tres Threads nuevos para cada simulación.
- Impide iniciar otra ronda mientras enEjecucion sea verdadero y mantiene cancelacionSolicitada.
- Recibe el avance de cada vendedor y forma un arreglo int[] de ventas individuales.
- Envía notificaciones mediante Platform.runLater(), que activan CajaController.actualizarAvanceVentas().
- En Cancelar interrumpe los tres Threads; un coordinador hace join() y finalmente permite el reinicio.

11.4 Trayectoria de los cuatro progresos

caja-view.fxml (barraVendedor1/2/3, barraGeneral) → CajaController.iniciarSimulacion() → CajaService.iniciarSimulacionVentas() → Vendedor.run() → Caja.registrarVentas() → ResultadoCaja → Platform.runLater() → CajaController.actualizarAvanceVentas() → setProgress() en la barra general y en cada vendedor.

12 / CASO 3 · PRUEBAS

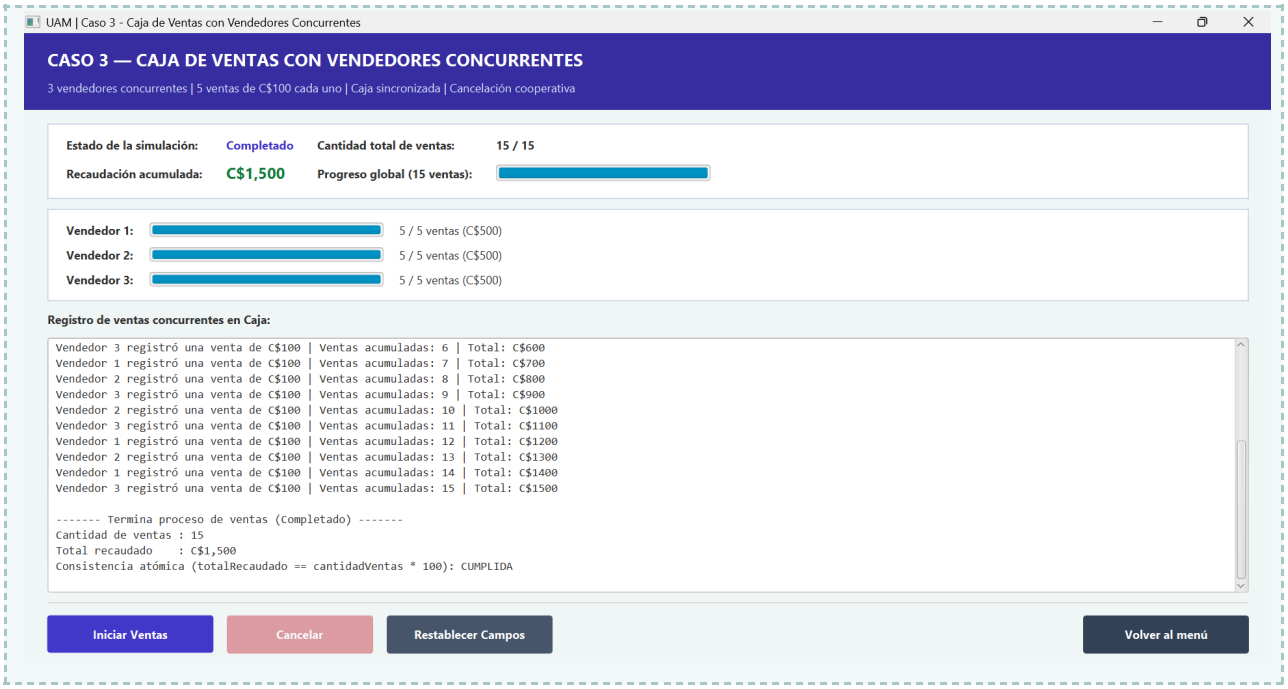
Resultados y evidencias de caja de ventas

La rúbrica exige cinco ejecuciones completas y pruebas adicionales de cancelación, reinicio, duplicados y cierre.

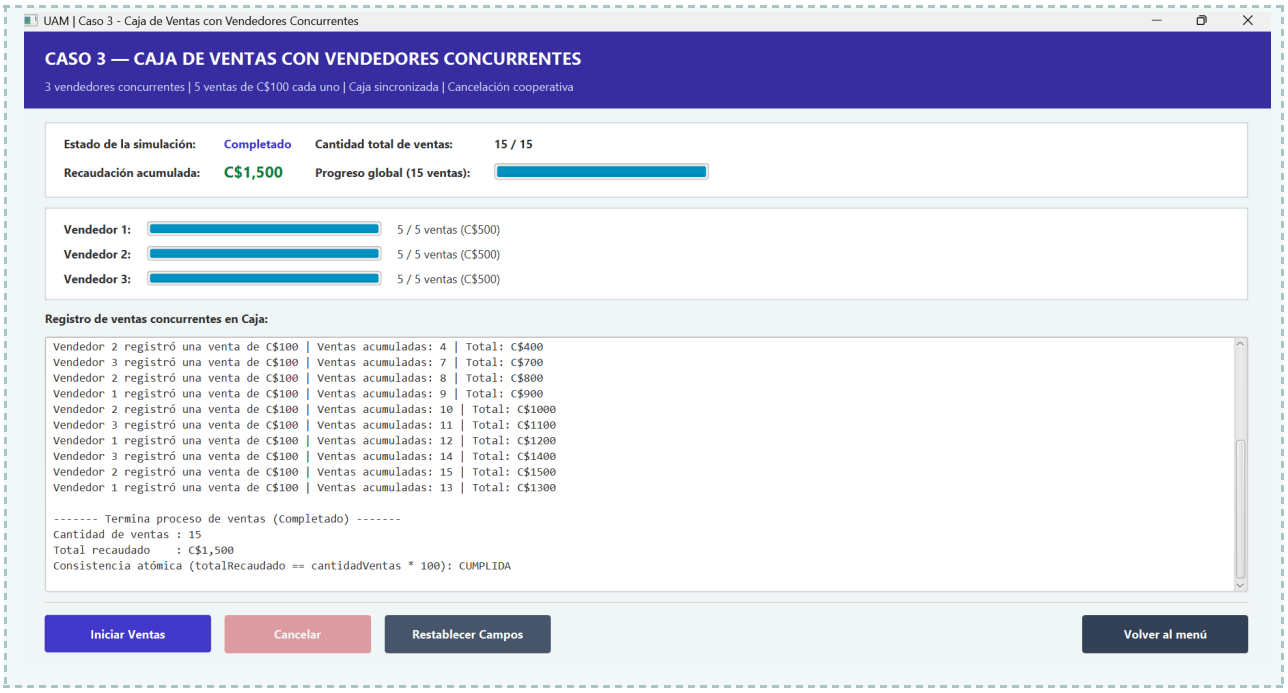
Prueba	Ventas esperadas	Recaudación esperada	Regla de consistencia	Observado / fecha
C1	15	C$1,500	monto = ventas × 100	Pendiente
C2	15	C$1,500	monto = ventas × 100	Pendiente
C3	15	C$1,500	monto = ventas × 100	Pendiente
C4	15	C$1,500	monto = ventas × 100	Pendiente
C5	15	C$1,500	monto = ventas × 100	Pendiente

En las cinco pruebas completas debe verse 5/5 ventas en cada vendedor, 15/15 ventas generales y C$1,500. La prioridad de la cancelación parcial es demostrar que el dinero corresponde exactamente a las ventas registradas aunque no lleguen a 15.

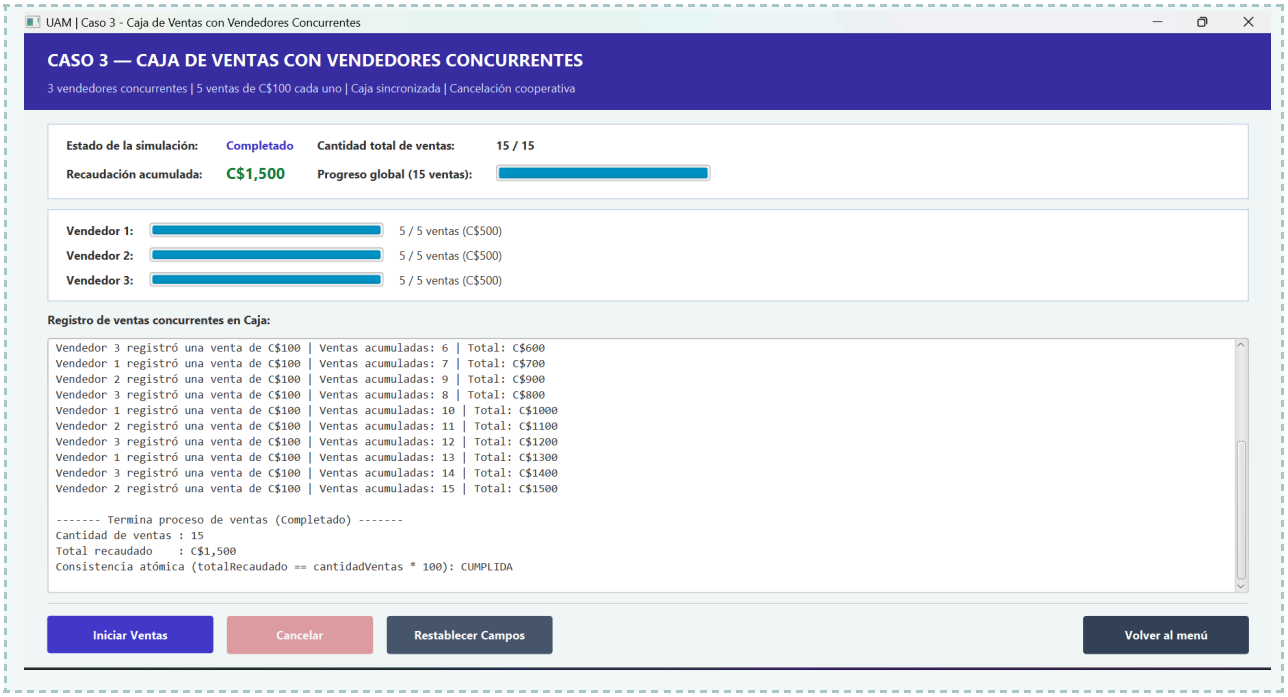
EVIDENCIA C1 · EJECUCIÓN COMPLETA NÚMERO 1



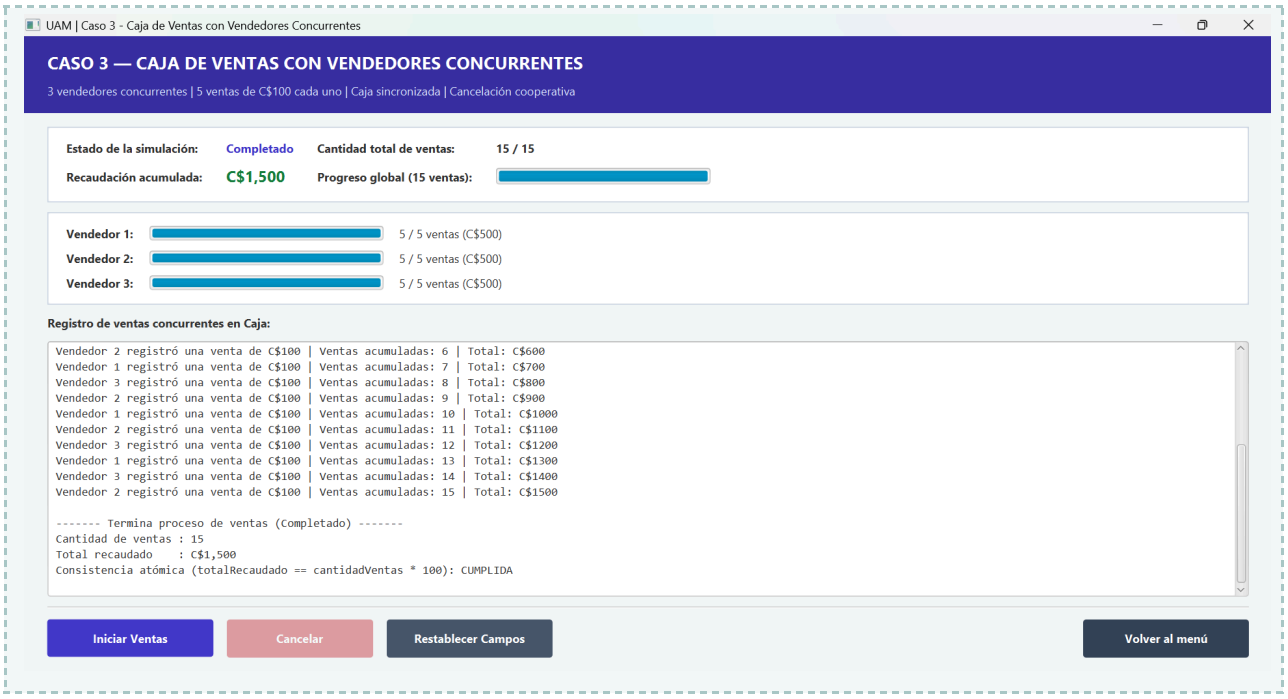
EVIDENCIA C2 · EJECUCIÓN COMPLETA NÚMERO 2



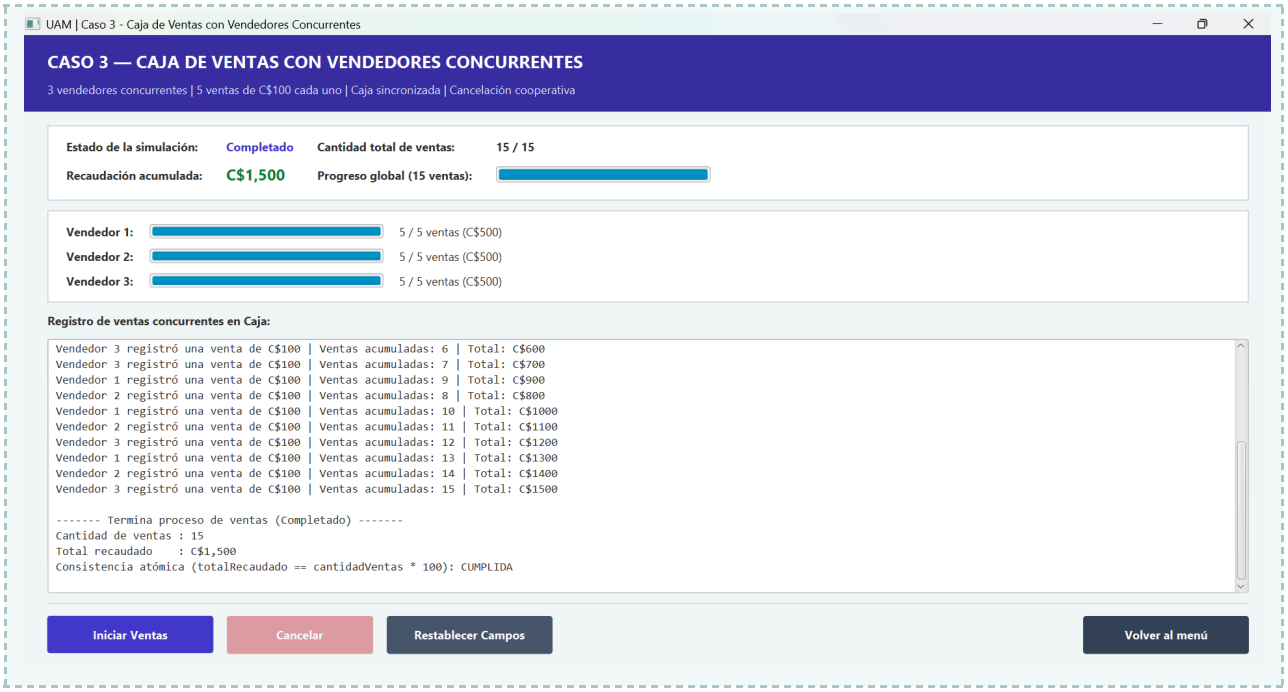
EVIDENCIA C3 · EJECUCIÓN COMPLETA NÚMERO 3



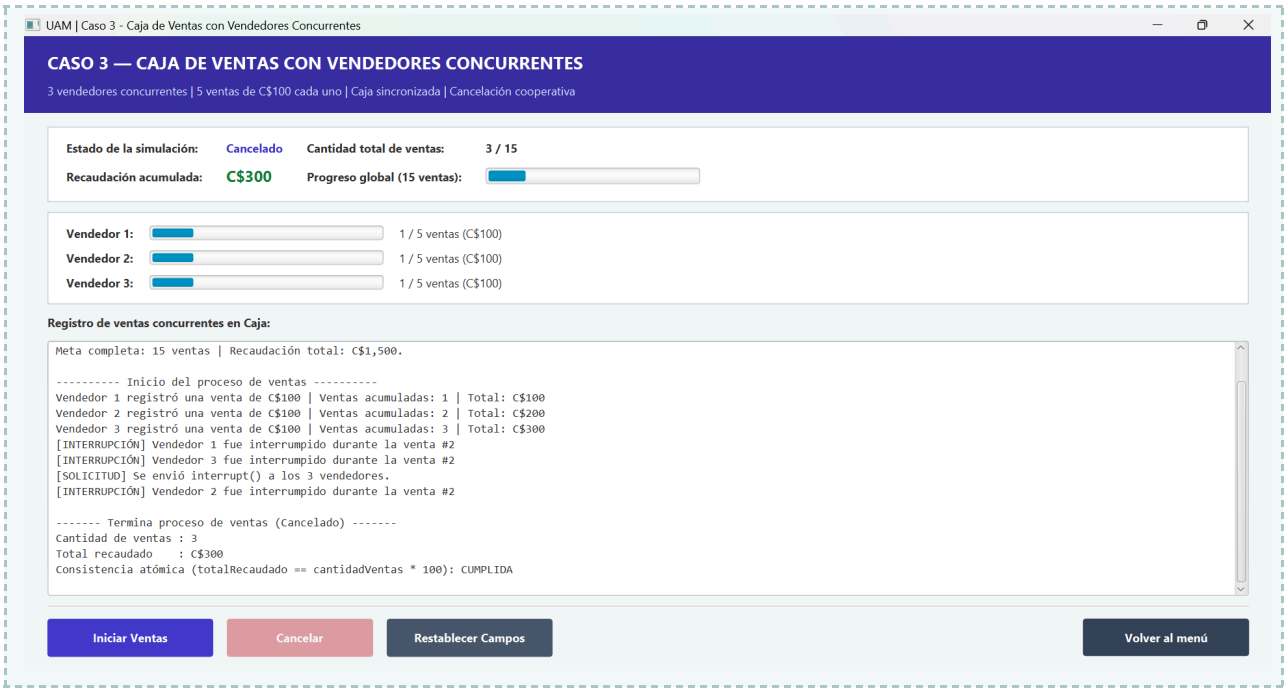
EVIDENCIA C4 · EJECUCIÓN COMPLETA NÚMERO 4



EVIDENCIA C5 · EJECUCIÓN COMPLETA NÚMERO 5

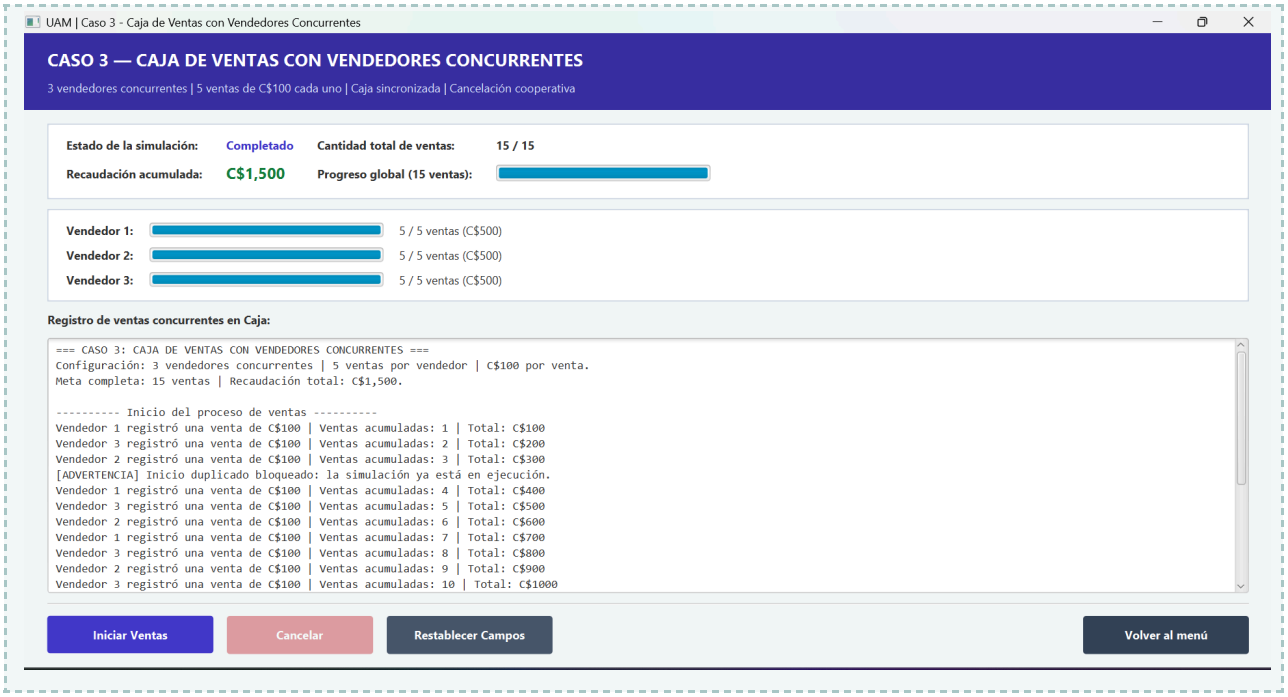


EVIDENCIA C6 · CANCELACIÓN PARCIAL

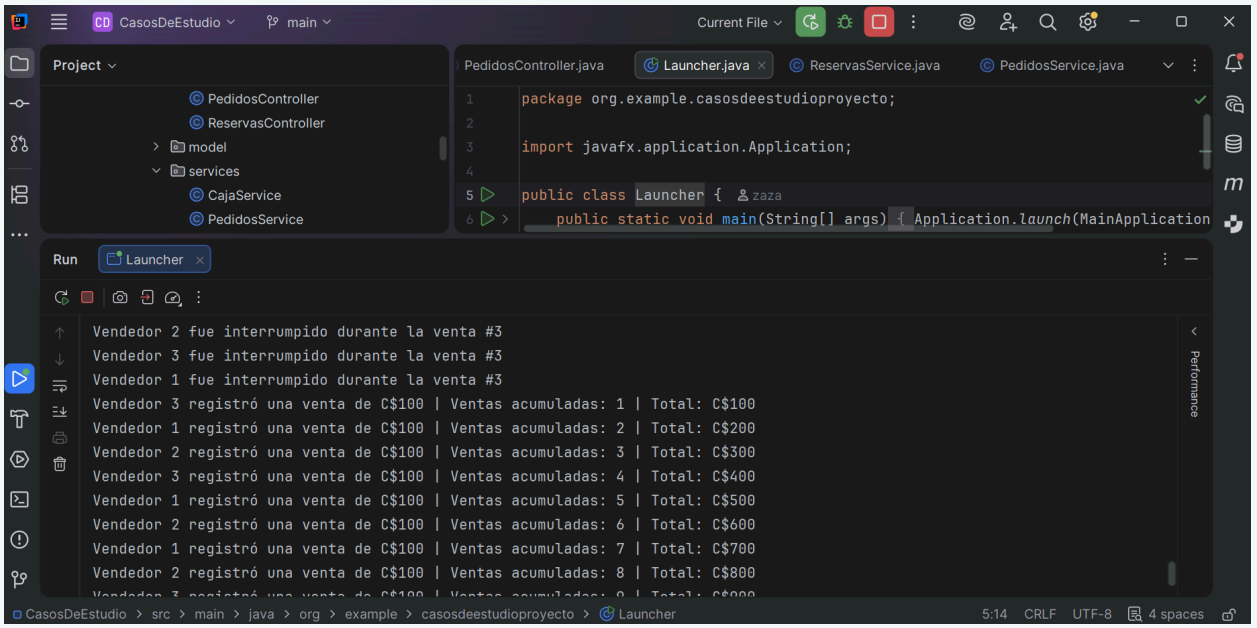


EVIDENCIA C7 · INICIO DUPLICADO

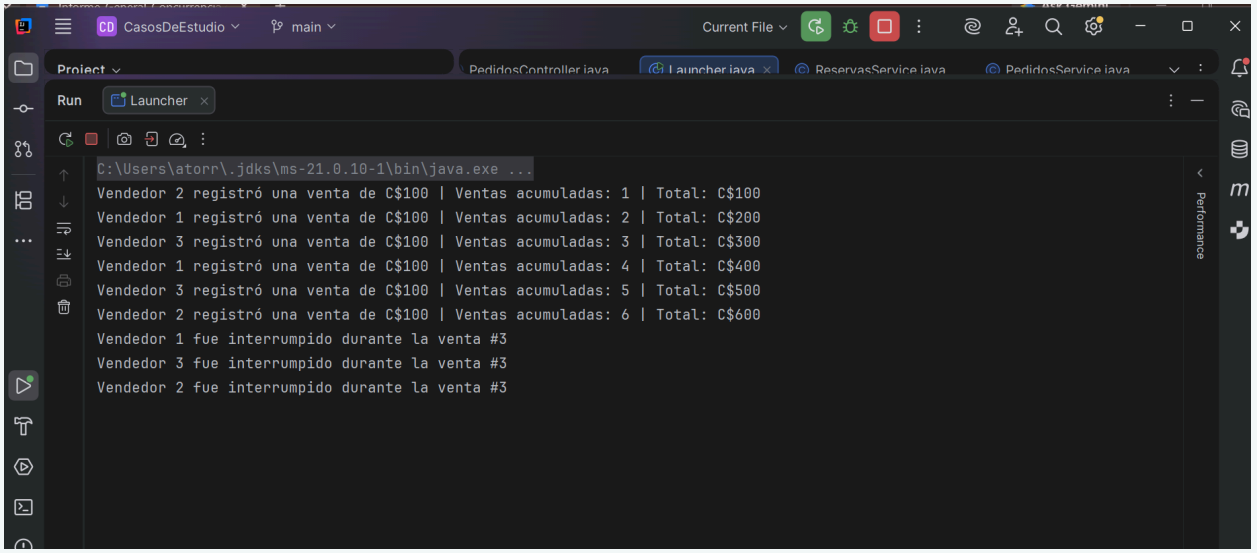
Captura que debes insertar: Capturar advertencia o evidencia de que no se crea una nueva simulación mientras están activos los vendedores.



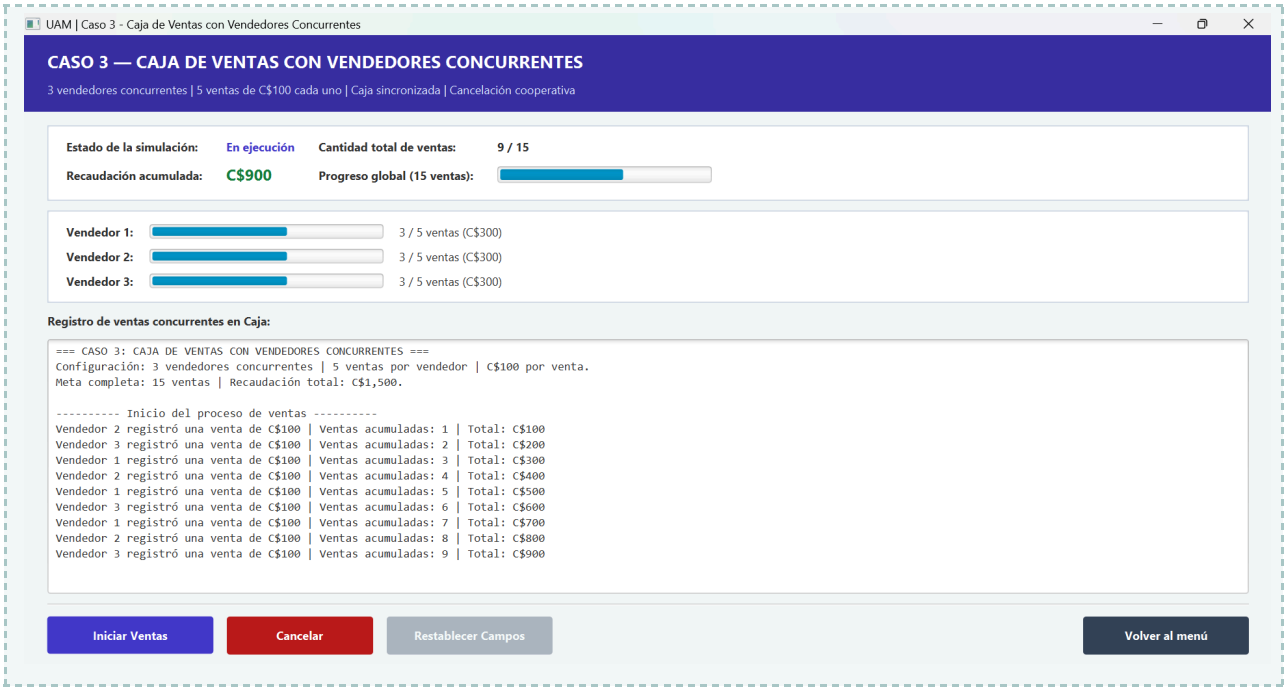
EVIDENCIA C8 · REINICIO DESPUÉS DE CANCELAR



EVIDENCIA C9 · CIERRE CON VENDEDORES ACTIVOS



EVIDENCIA C10 · PROGRESO INDIVIDUAL SIMULTÁNEO



13 / RÚBRICAS

Matriz de criterios de evaluación por caso

Cada tarea tiene cinco criterios de ocho puntos. Los documentos y capturas deben respaldar cada criterio.

Caso 1	Qué debemos demostrar para aspirar a 8 pts	Evidencia
Concurrencia	Tres agentes simultáneos y rechazo de duplicados	R6, R7
Sincronización	Una sola sección crítica; inventario no negativo	Código 09, R1–R5
Resultados y reglas	20/10/0 durante cinco pruebas	R1–R5
Interfaz JavaFX	Tres resultados y barra operativa sin bloqueo	Código 07–08, R7
Pruebas y colaboración	Cinco pruebas, commits y exposición de tres integrantes	R1–R8 + tabla de roles
Caso 2	**Qué debemos demostrar para aspirar a 8 pts**	**Evidencia**
Segundo plano	30 pedidos fuera del hilo JavaFX	P1
Cancelación	interrupt() comprobado; no sumar pedido incompleto	Código 12, P2
Ciclo de vida	Sin duplicados; reinicio/cierre tras terminación	P3, P4, P5
Progreso y estados	Contador, barra y estados coherentes	Código 11, P1–P2
Pruebas y colaboración	Escenarios completos + aportes verificables	P1–P5
Caso 3	**Qué debemos demostrar para aspirar a 8 pts**	**Evidencia**
Tareas concurrentes	Tres vendedores, cinco ventas por cada uno	C1–C5
Recursos compartidos	Venta y dinero atómicos, incluso al cancelar	Código 17, C6
Ciclo de vida	Cancelación de los tres, reinicio y cierre	C6–C9

Interfaz	Barras individual/general y 15/C$1,500	Código 14–16, C1–C5
Validación grupal	Cinco pruebas, otros escenarios y defensa grupal	C1–C10

13.1 Trazabilidad de evidencias

Las capturas de pantalla muestran el estado visual de una ejecución, pero no prueban por sí solas la ausencia de hilos vivos. Para cierre y cancelación es recomendable acompañarlas con el registro de consola o una explicación del código de coordinación. Tampoco basta con mostrar una sola ejecución de Reservas o Caja: las rúbricas exigen cinco.

14 / ENTREGA

Pruebas manuales, colaboración y documento final

Campos preparados para completar con evidencias reales antes de entregar en UAM Virtual.

14.1 Cómo ejecutar y comprobar

- Abrir CasosDeEstudioProyecto en IntelliJ IDEA, configurar el SDK de Java 21 y sincronizar Maven.
- Ejecutar mvn clean javafx:run desde la raíz que contiene pom.xml.
- En cada pantalla, capturar estado inicial, progreso intermedio y resultado final según las pruebas.
- Mantener evidencia individual de las cinco ejecuciones de Reservas y las cinco de Caja.
- Probar Cancelar y cierre durante ejecución en Pedidos y Caja; probar volver al menú con actividad.
- Verificar que las ventanas responden y que los contadores respetan los resultados esperados.
- Insertar capturas en los recuadros y registrar fecha, resultado real y responsable de la prueba.

14.2 Conclusión integradora

Los tres casos permiten comprender la relación entre concurrencia, sincronización, cancelación y desarrollo de interfaces JavaFX. El elemento visual más visible —la barra de progreso— es el resultado de varias capas trabajando juntas: el FXML define el control, el controller lo actualiza, los services entregan resultados desde hilos secundarios y los models ejecutan las operaciones. El mismo diseño evita mezclar trabajo pesado con el hilo gráfico, permite ejecutar nuevas simulaciones y mantiene los recursos compartidos consistentes.

La documentación queda lista para integrar pruebas reales. La ejecución completa, el comportamiento al cancelar y el cierre seguro deben confirmarse con capturas y registros en IntelliJ antes de declarar cumplidos los criterios de la rúbrica.

14.3 Fuentes de la documentación

- Rúbricas de las tareas sumativas 1, 2 y 3 de Programación de Aplicaciones de Escritorio (capturas proporcionadas).
- Código de la rama zahid y alessandro del repositorio CasosDeEstudio: https://github.com/rzahidb/CasosDeEstudio/tree/zahid (consultado el 28/10/2026).